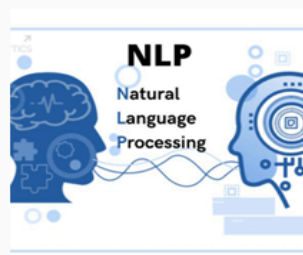


Chapter 12: NLP — Natural Language Processing

INTRODUCTION

Natural Language Processing is a subset of artificial intelligence that enables machines to comprehend and analyze human languages. Text or audio can be used to represent human languages.



1. NLP কী? (What is NLP?)

NLP (Natural Language Processing) হলো AI এর একটা subset যেটা machines কে human language বুঝতে এবং analyze করতে সাহায্য করে।

Simple কথায়: Computer কে শেখানো যে সে যেন মানুষের ভাষা বুঝতে পারে — text হোক বা audio।

Real life examples:

- ChatGPT, Siri, Alexa → তোমার কথা বোঝে
- Google Translate → এক ভাষা থেকে আরেক ভাষায় translate করে
- Gmail Spam Filter → কোনটা spam সেটা বোঝে
- Sentiment Analysis → review পড়ে বোঝে positive না negative

2. NLP এর Goals

Goal	মানে
Convert human language → machine format	মানুষের ভাষাকে computer বোঝার উপযোগী করা
Extract meaning & patterns	text থেকে meaning বের করা
Enable communication	machine যেন মানুষের সাথে কথা বলতে পারে

WHY NLP MATTERS

- NLP enables machines to interpret text and speech which are critical for search, chatbots, translators, summarizers, etc.. For example, chat assistants, spam filters, and voice interfaces rely on NLP.
- Data abundance: Over 80% of enterprise data is unstructured text (emails, social media, reports), so NLP unlocks business insights.
- Cross-domain impact: From healthcare (clinical notes) to finance (document analysis) to social media (sentiment).
- Trends: Deep learning and large pre-trained models (BERT, GPT) have dramatically boosted NLP capabilities

3. Why NLP Matters? (কেন NLP দরকার?)

৪টা কারণ মনে রাখো:

① Text is everywhere:

৮০% এরও বেশি enterprise data হলো unstructured text — emails, social media, reports। NLP এই data থেকে মূল্যবান তথ্য বের করে।

② Cross-domain impact:

-  Healthcare → doctor এর clinical notes analyze করা
-  Finance → document analysis
-  Social Media → sentiment বোঝা

③ Modern models are powerful:

BERT, GPT এর মতো large pre-trained models NLP কে অনেক শক্তিশালী করেছে।

④ Everything depends on it:

Search engine, chatbot, voice assistant — সবকিছুর পেছনে NLP।

LINGUISTIC FUNDAMENTALS

- **Morphology:** Words are built from morphemes (smallest meaning units: roots, prefixes, suffixes).
- **Part-of-Speech (POS):** Each word falls into categories (noun, verb, etc.), which affects meaning. POS tagging assigns these categories, aiding syntactic parsing and disambiguation.
- **Syntax:** Grammar rules govern sentence structure (e.g. "The cat sat" vs. "Cat the sat"). Syntax includes parse trees and dependencies. Later used in advanced parsing models.
- **Semantics:** Concerned with meaning. Words can have multiple senses. Context and semantic networks are key for understanding synonyms/antonyms.
- **Impact on NLP:** Many feature extraction and model choices (e.g. lemmatizer vs. stemmer) derive from these linguistic concepts.

4. Linguistic Fundamentals (ভাষার মূল ধারণাগুলো)

NLP বোঝার আগে ভাষা সম্পর্কে কিছু basic জানতে হবে:

Morphology (মর্ফোলজি)

Definition: Words are built from **morphemes** — the smallest units of meaning.

"unhappiness"

= un (prefix) + happy (root) + ness (suffix)

↓ ↓ ↓
নেই/না সুখী অবস্থা

মানে: morpheme হলো শব্দের সবচেয়ে ছোট অর্থবহ অংশ।

Part-of-Speech (POS) Tagging

Definition: প্রতিটি word কে তার grammatical category (noun, verb, adjective...) অনুযায়ী tag করা।

"The cat sat on the mat"

↓ ↓ ↓ ↓ ↓ ↓
DET NOUN VERB PREP DET NOUN

কেন দরকার? → এটা sentence এর structure বুঝতে সাহায্য করে।

Syntax (সিনট্যাক্স)

Definition: Grammar rules যা sentence এর structure নির্ধারণ করে।

✓ "The cat sat" → grammatically correct

✗ "Cat the sat" → grammatically wrong

Syntax → parse trees এবং dependencies তৈরি করে।

Semantics (সিম্যান্টিক্স)

Definition: শব্দের meaning নিয়ে কাজ করে।

"Bank" শব্দটার দুটো মানে হতে পারে:

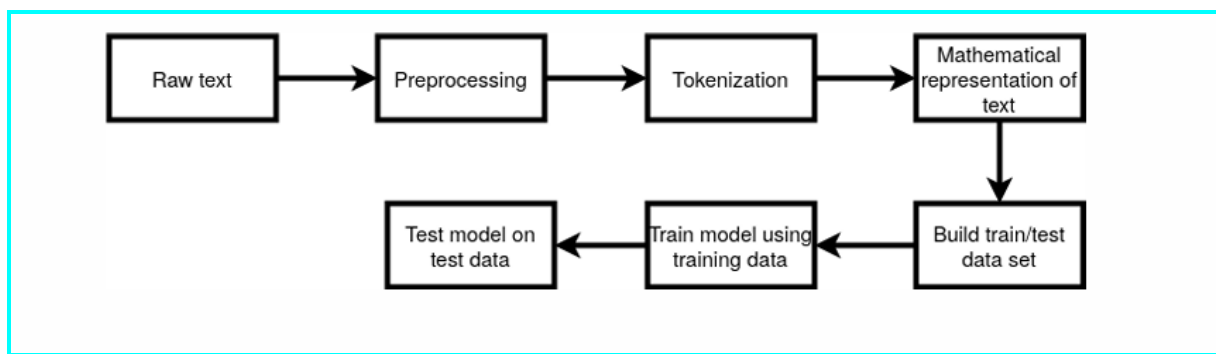
→ নদীর পাড় (river bank)

→ টাকা রাখার জায়গা (financial bank)

Context দিয়ে সঠিক মানে বোঝা হয় — এটাই semantics।

Impact on NLP: Many feature extraction and model choices (e.g. lemmatizer vs. stemmer) derive from these linguistic concepts

NLP-তে একটি শব্দকে তার মূল রূপে ফিরিয়ে নেওয়ার জন্য **Stemming** (শব্দের শেষের অংশ ছেঁটে ফেলা) বা **Lemmatization** (অভিধান অনুযায়ী মূল শব্দ খোঁজা) ব্যবহার করা হয়। এই টেকনিকগুলো মূলত লিঙ্গুইস্টিক বা ভাষাতাত্ত্বিক নিয়মের ওপর ভিত্তি করে তৈরি, যা কম্পিউটারকে শব্দের অর্থ এবং ব্যাকরণ বুঝতে সাহায্য করে। সহজ কথায়, সঠিক ভাষাতাত্ত্বিক ধারণা ব্যবহার করলে মডেলের পারফরম্যান্স অনেক বেশি নির্ভুল হয়।



EXAMPLE SENTENCE PROCESSING

Input Sentence: "I love this movie so much!"

1. Raw Text: The original sentence provided by the user.

Result: "I love this movie so much!"

2. Preprocessing: Cleaning the text for easier processing.

Steps:

- Convert to lowercase
- Remove punctuation

Result: i love this movie so much

3. Tokenization: Split the sentence into individual tokens.

Result: ["i", "love", "this", "movie", "so", "much"]

Why it is important

- Makes text easier to process
- Helps later steps like tagging and vectorization
- Preserves structure for the model

4. Mathematical Representation: Convert words into numerical vectors.

Computers cannot work directly with words, so text must be converted into numbers.

Common representations

- Bag-of-Words
- TF-IDF
- Word embedding

5. NLP Pipeline (ধাপে ধাপে প্রক্রিয়া)

এই ছবিটিতে একটি **NLP (Natural Language Processing)** প্রজেক্ট বা পাইপলাইনের জীবনচক্র দেখানো হয়েছে। কম্পিউটার কীভাবে মানুষের ভাষা (Text) নিয়ে কাজ করে এবং একটি মডেল তৈরি করে, তা এখানে ধাপে ধাপে বর্ণনা করা হয়েছে।

একটি উদাহরণ দিয়ে প্রতিটি ধাপ বুঝে নেওয়া যাক। ধরো, আমরা একটি মডেল বানাবো যা ইমেইল পড়ে বলবে সেটি **"Spam"** (ভুল ইমেইল) নাকি **"Ham"** (সঠিক ইমেইল)।

১. Raw Text (কাঁচা তথ্য)

এটি হলো একদম শুরুর ধাপ যেখানে ডাটাগুলো অগোছালো অবস্থায় থাকে।

- উদাহরণ: আমাদের কাছে ইমেইল আসলো— *"FREE!!! Click here to win \$1000 cash now!!!"*

২. Preprocessing (প্রাক-প্রক্রিয়াকরণ)

কম্পিউটার অপ্রয়োজনীয় জিনিস পছন্দ করে না। এই ধাপে টেক্সট পরিষ্কার করা হয়।

- কাজ: ছোট হাতের অক্ষর করা (Lowercase), বিরামচিহ্ন (Punctuation) বাদ দেওয়া এবং স্টপ-ওয়ার্ডস (is, the, to) বাদ দেওয়া।
- উদাহরণ: *"free click win 1000 cash"* (বাকি সব বাদ দেওয়া হয়েছে)।

৩. Tokenization (টোকেনাইজেশন)

পুরো বাক্যকে ছোট ছোট টুকরো বা 'টোকেন'-এ ভাগ করা হয়। সাধারণত প্রতিটি শব্দ একটি টোকেন।

- উদাহরণ: ["free", "click", "win", "1000", "cash"]

৪. Mathematical Representation of Text (গাণিতিক রূপান্তর)

কম্পিউটার শব্দ বোঝে না, সে শুধু সংখ্যা (Numbers) বোঝে। তাই শব্দগুলোকে ভেক্টর বা সংখ্যায় রূপান্তর করতে হয়।

- কাজ: **TF-IDF** বা **Word2Vec** পদ্ধতি ব্যবহার করা।
- উদাহরণ: "free" শব্দটিকে হয়তো সংখ্যায় লেখা হলো $[1, 0, 0]$, "click" হলো $[0, 1, 0]$ ইত্যাদি।

৫. Build Train/Test Data Set (ডেটা ভাগ করা)

তোমার কাছে যত ইমেইল আছে, সব দিয়ে কিন্তু ট্রেনিং করানো হয় না। কিছু ডাটা আলাদা রাখা হয় মডেলকে পরীক্ষা করার জন্য।

- উদাহরণ: ৮০% ইমেইল দিয়ে আমরা মডেলকে **Train** করাবো এবং ২০% ইমেইল দিয়ে পরে তাকে **Test** করবো।

৬. Train Model using Training Data (মডেল ট্রেনিং)

এই ধাপে আমরা অ্যালগরিদমকে (যেমন: Naive Bayes বা Logistic Regression) ডাটাগুলো দেখাই। সে শেখে যে "free", "win", "cash" শব্দগুলো থাকলে সেটা সাধারণত স্প্যাম হয়।

- উদাহরণ: মডেল শিখছে: যদি ইমেইলে "win" থাকে, তবে এটি ৯৫% নিশ্চিতভাবে স্প্যাম।

৭. Test Model on Test Data (মডেল পরীক্ষা)

সবশেষে, ওই ২০% ডাটা যা মডেল আগে কখনো দেখেনি, তা তাকে দেওয়া হয়। দেখা হয় সে কতটুকু নির্ভুলভাবে উত্তর দিতে পারছে।

- উদাহরণ: মডেলকে একটি নতুন ইমেইল দেওয়া হলো, সে সঠিকভাবে বলতে পারল কি না যে এটি স্প্যাম।

সহজ সারকথা: প্রথমে টেক্সট নেওয়া হয় (**Raw Text**), সেটি পরিষ্কার করা হয় (**Preprocessing**), টুকরো করা হয় (**Tokenization**), সংখ্যায় রূপান্তর করা হয় (**Math Representation**), দুই ভাগে ভাগ করা হয় (**Split**), কম্পিউটারকে শেখানো হয় (**Train**) এবং সবশেষে দেখা হয় সে কতটুকু শিখল (**Test**)।

The Bag of Words (BoW) model is the simplest form of text representation in numbers. Like the term itself, we can represent a sentence as a bag of words vector (a string of numbers).

Let's discuss three types of movie reviews:
 Review 1: This movie is very scary and long
 Review 2: This movie is not scary and is slow
 Review 3: This movie is spooky and good

We will first build a vocabulary from all the unique words in the above three reviews. The vocabulary consists of these 11 words: 'This', 'movie', 'is', 'very', 'scary', 'and', 'long', 'not', 'slow', 'spooky', 'good'.

We can now take each of these words and mark their occurrence in the three movie reviews above with 1s and 0s. This will give us 3 vectors for 3 reviews:

	1 This	2 movie	3 is	4 very	5 scary	6 and	7 long	8 not	9 slow	10 spooky	11 good	Length of the review(in words)
Review 1	1	1	1	1	1	1	1	0	0	0	0	7
Review 2	1	1	2	0	0	1	1	0	1	0	0	8
Review 3	1	1	1	0	0	0	1	0	0	1	1	6

Vector-of-Review1: [1 1 1 1 1 1 1 0 0 0 0]

Vector-of-Review2: [1 1 2 0 0 1 1 0 1 0 0]

Vector-of-Review3: [1 1 1 0 0 0 1 0 0 1 1]

And that's the core idea behind a Bag of Words (BoW) model.

However, we start facing issues of Bag of Words when we come across new sentences:

- If the new sentences contain new words, then our vocabulary size would increase and thereby, the length of the vectors would increase too.
- Additionally, the vectors would also contain many 0s, thereby resulting in a sparse matrix (which is what we would like to avoid)
- We are retaining no information on the grammar of the sentences nor on the ordering of the words in the text.

6. Bag of Words (BoW) — বিস্তারিত

Definition: Text কে numbers এ represent করার সবচেয়ে সহজ পদ্ধতি — প্রতিটি sentence কে একটা vector এ রূপান্তর করা হয়।

উদাহরণ — ৩টা **movie review**:

Review 1: "This movie is very scary and long"

Review 2: "This movie is not scary and is slow"

Review 3: "This movie is spooky and good"

Step 1: সব unique word দিয়ে vocabulary বানাও:

['This', 'movie', 'is', 'very', 'scary', 'and',
'long', 'not', 'slow', 'spooky', 'good']

↑ মোট ১১টা unique word

Step 2: প্রতিটি review তে কোন word কতবার আছে সেটা count করো:

This movie is very scary and long not slow spooky good

Review 1: 1 1 1 1 1 1 1 0 0 0 0

Review 2: 1 1 2 0 0 1 1 0 1 0 0

Review 3: 1 1 1 0 0 0 1 0 0 1 1

Vectors:

Review 1 → [1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0]

Review 2 → [1, 1, 2, 0, 0, 1, 1, 0, 1, 0, 0]

Review 3 → [1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1]

! BoW এর সমস্যা:

✗ নতুন word এলে vocabulary বড় হয়ে যায়

✗ অনেক 0 থাকে → sparse matrix তৈরি হয়

✗ Word এর order বা grammar বোঝে না

("I love you" আর "you love I" একই হয়ে যায়!)

ব্যাগ অফ ওয়ার্ডস (BoW) পদ্ধতিতে সাধারণত ক্যাপিটাল এবং স্মল লেটারকে আলাদা দুটি শব্দ হিসেবেই কাউন্ট করা হয় (যেমন: 'Apple' এবং 'apple')। তবে প্রসেসিংয়ের সময় যদি **Lowercasing** (সব ছোট হাতের করা) করে নেওয়া হয়, তবেই কেবল সেটিকে একটি শব্দ হিসেবে গণ্য করা হবে।

TF-IDF (Term Frequency-Inverse Document Frequency) is a statistical method used in natural language processing and information retrieval to evaluate how important a word is to a document in relation to a larger collection of documents. TF-IDF combines two components:

1. **Term Frequency (TF):** Measures how often a word appears in a document. A higher frequency suggests greater importance. If a term appears frequently in a document, it is likely relevant to the document's content.

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

2. **Inverse Document Frequency (IDF):** Reduces the weight of common words across multiple documents while increasing the weight of rare words. If a term appears in fewer documents, it is more likely to be meaningful and specific.

$$IDF(t, D) = \log \frac{\text{Total number of documents in corpus } D}{\text{Number of documents containing term } t}$$

$$TF - IDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

7. TF-IDF — বিস্তারিত

Full form: Term Frequency — Inverse Document Frequency

Definition: একটা statistical method যা বলে কোনো word একটা document এর জন্য কতটা গুরুত্বপূর্ণ।

 দুইটা অংশ:

① TF (Term Frequency):

কোনো word একটা document এ কতবার আছে।

$TF(t, d) = (\text{document } d \text{ তে term } t \text{ কতবার আছে}) \div (\text{document } d \text{ তে মোট word সংখ্যা})$

না, TF (Term Frequency) এর ফর্মুলায় **Total number of words** বলতে মোট যতগুলো শব্দ আছে (Total count), সবগুলোকে বোঝায়; শুধু ইউনিক (Unique) শব্দগুলোকে নয়।

Example:

Document: "the cat sat on the mat" → মোট 6 words

"the" আছে 2 বার

$TF("the") = 2/6 = 0.33$

"cat" আছে 1 বার

$TF("cat") = 1/6 = 0.17$

② IDF (Inverse Document Frequency):

যে word সব document এ আছে (যেমন "the", "is") সেটা কম important। যে word কম document এ আছে সেটা বেশি specific ও গুরুত্বপূর্ণ।

$IDF(t, D) = \log(\text{মোট document সংখ্যা} \div \text{যত document এ } t \text{ আছে})$

Example:

মোট 3টা document আছে।

"scary" আছে মাত্র 1টা document এ:

$IDF("scary") = \log(3/1) = \log(3) \approx 0.48$

"movie" আছে 3টা document এ:

$IDF("movie") = \log(3/3) = \log(1) = 0$

↑ "movie" সব জায়গায় আছে তাই এর importance = 0

Final Formula:

$TF-IDF(t, d, D) = TF(t, d) \times IDF(t, D)$

TF-IDF

Applications:

1. **Document Similarity and Clustering:** By converting documents into numerical vectors TF-IDF enables comparison and grouping of related texts. This is valuable for clustering news articles, research papers or customer support tickets into meaningful categories.
2. **Text Classification:** It helps in identify patterns in text for spam filtering, sentiment analysis and topic classification.
3. **Keyword Extraction:** It ranks words by importance making it possible to automatically highlight key terms, generate document tags or create concise summaries.
4. **Recommendation Systems:** Through comparison of textual descriptions TF-IDF supports suggesting related articles, videos or products enhancing user engagement.

TF-IDF এর Applications:

TF-IDF (Term Frequency-Inverse Document Frequency) এর এই অ্যাপ্লিকেশনগুলো খুবই গুরুত্বপূর্ণ। সহজভাবে বললে, TF-IDF আমাদের বলে দেয় একটি বড় লাইব্রেরিতে বা অনেকগুলো ফাইলের মধ্যে কোন শব্দটি আসলে "গুরুত্বপূর্ণ"।

নিচে এর ব্যবহারগুলো সহজ বাংলায় বুঝিয়ে দিচ্ছি:

১. Document Similarity and Clustering (মিল খুঁজে বের করা)

এটি দুটি ডকুমেন্টের মধ্যে মিল খুঁজে পেতে সাহায্য করে।

- কীভাবে কাজ করে: ধরো তোমার কাছে ১০০০টি নিউজ আর্টিকেল আছে। TF-IDF প্রতিটি আর্টিকেলকে সংখ্যার একটি ভেক্টরে (Vector) রূপান্তর করে। এখন কম্পিউটার অংক কষে দেখতে পারে কোন দুটি ভেক্টরের মান কাছাকাছি।
- উদাহরণ: খেলাধুলার সব খবর একদিকে, আর রাজনীতির খবরগুলো অন্যদিকে আলাদা গ্রুপ বা **Cluster** হয়ে যাবে। কাস্টমার সাপোর্ট টিকেটগুলোকেও এভাবে অটোম্যাটিক আলাদা করা যায় (যেমন: বিলিং সমস্যা বনাম টেকনিক্যাল সমস্যা)।

২. Text Classification (শ্রেণীবদ্ধ করা)

কোনো টেক্সট কোন ক্যাটাগরির, তা বুঝতে এটি দারুণ কাজ করে।

- কীভাবে কাজ করে: এটি টেক্সটের প্যাটার্ন বা ধরন চিনে ফেলে।
- উদাহরণ: **Spam Filtering**-এর ক্ষেত্রে এটি দেখে "WIN", "CASH", "FREE" শব্দগুলোর গুরুত্ব যদি ইমেইলে খুব বেশি হয়, তবে সেটাকে 'স্প্যাম' হিসেবে ক্লাসিফাই করে দেয়। আবার সোশ্যাল মিডিয়া কमेंট দেখে **Sentiment Analysis** করতে পারে (যেমন: কमेंটটি পজিটিভ না নেগেটিভ)।

৩. Keyword Extraction (মূল শব্দ খুঁজে বের করা)

একটি বিশাল আর্টিকেল থেকে সবচেয়ে দরকারি শব্দগুলো বের করে আনা।

- কীভাবে কাজ করে: আমরা জানি সাধারণ শব্দ (যেমন: 'is', 'the') সবখানে থাকে, তাই এদের গুরুত্ব কম। কিন্তু কোনো বিশেষ শব্দ যদি শুধু একটি নির্দিষ্ট ডকুমেন্টে বারবার আসে, TF-IDF সেটাকে হাই র্যাঙ্ক (High Rank) দেয়।

- উদাহরণ: একটি হার্ট অ্যাটাক নিয়ে লেখা রিসার্চ পেপার থেকে এটি "Myocardial Infarction" বা "Artery" শব্দগুলোকে **Keyword** হিসেবে টেনে বের করবে, যা দিয়ে আপনি অটোমেটিক ট্যাগ (Tag) বা সামারি তৈরি করতে পারবেন।

8. Recommendation Systems (পরামর্শ দেওয়া)

তোমার পছন্দের জিনিসের সাথে মিল রেখে অন্য কিছু সাজেস্ট করা।

- কীভাবে কাজ করে: তুমি যদি একটি আর্টিকেল পড়ো যার ডেসক্রিপশনে নির্দিষ্ট কিছু কী-ওয়ার্ড আছে, সিস্টেম TF-IDF ব্যবহার করে ড্যাটাবেজে থাকা অন্য আর্টিকেলগুলোর সাথে তুলনা করে।
- উদাহরণ: ইউটিউবে তুমি যখন একটি রান্নার ভিডিওর ডেসক্রিপশন পড়ো বা ভিডিও দেখো, সিস্টেম TF-IDF দিয়ে দেখে অন্য কোন ভিডিওর ডেসক্রিপশন এটার সাথে মিলছে, এবং তখন তোমাকে সেই ভিডিওটি **Recommend** করে।

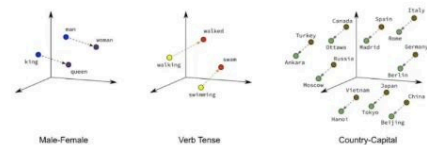
সহজ সারাংশ (Intuition):

- Similarity:** একই ধরনের ফাইলগুলো এক করা।
- Classification:** ফাইলটির নাম বা লেবেল দেওয়া (যেমন: স্প্যাম)।
- Keyword:** ফাইলের সবচেয়ে দরকারি শব্দ বের করা।
- Recommendation:** এক ফাইলের সাথে মিল রেখে অন্য ফাইল খুঁজে দেওয়া।

Word Embeddings

Definition

- Word embeddings are numerical vector representations of words
- Words are placed in a multi-dimensional space
- Similar words are located closer to each other
- They capture semantic meaning and relationships between words



Why Word Embeddings Are Needed

- Computers cannot understand words directly
- Traditional methods like one-hot encoding create sparse vectors
- Word embeddings create dense vectors with meaningful values
- They represent words in a smarter and more useful way

How They Work

- Words are converted into dense vectors
- These vectors are learned from large text data
- Words used in similar contexts get similar vector representations
- This helps the model understand similarity and meaning

Word Embedding

Word embeddings are a way of representing words as vectors in a multi-dimensional space, where the distance and direction between vectors reflect the similarity and relationships among the corresponding words.

The development of embedding to represent text has played a crucial role in advancing natural language processing (NLP) and machine learning (ML) applications. Word embeddings have become integral to tasks such as text classification, sentiment analysis, machine translation and more.

Traditional methods of representing words in a way that machines can understand, such as one-hot encoding, represent each word as a sparse vector with a dimension equal to the size of the vocabulary. Here, only one element of the vector is "hot" (set to 1) to indicate the presence of that word.

8. Word Embedding — বিস্তারিত

Definition: Words কে **multidimensional space** এ vectors হিসেবে represent করা, যেখানে similar words একে অপরের কাছাকাছি থাকে। Word Embeddings হলো শব্দকে সংখ্যায় (Vectors) রূপান্তর করার আধুনিক স্মার্ট পদ্ধতি। এটি কেবল শব্দ গুলে রাখে না, বরং শব্দের অর্থ (**Meaning**) এবং সম্পর্ক (**Relationship**) বুঝতে পারে।

পুরোনো পদ্ধতি — **One-Hot Encoding** এর সমস্যা:

Vocabulary: [cat, dog, fish, bird]

cat → [1, 0, 0, 0]

dog → [0, 1, 0, 0]

fish → [0, 0, 1, 0]

bird → [0, 0, 0, 1]

সমস্যা: "cat" আর "dog" এর মধ্যে কোনো সম্পর্ক বোঝা যাচ্ছে না — দুটোই animal কিন্তু vectors এ সেটা নেই।

Word Embedding এর সুবিধা:

king → [0.9, 0.1, 0.8, ...]

queen → [0.9, 0.9, 0.8, ...]

man → [0.1, 0.1, 0.8, ...]

woman → [0.1, 0.9, 0.8, ...]

king - man + woman ≈ queen ✓

→ Mathematical relationship ধরতে পারছে!

Two approaches to word embeddings

1. Frequency-based embeddings: Frequency-based embeddings refer to word representations that are derived from the frequency of words in a corpus. These embeddings are based on the idea that the importance or significance of a word can be inferred from how frequently it occurs in the text.

2. Prediction-based embeddings: Prediction-based embeddings are word representations derived from models that are trained to predict certain aspects of a word's context or neighboring words. Unlike frequency-based embeddings that focus on word occurrence statistics, prediction-based embeddings capture semantic relationships and contextual information, providing richer representations of word meanings.

skhter900@gmail.com

18

দুইটা Approach:

১. Frequency-based Embeddings (ফ্রিকোয়েন্সি-ভিত্তিক)

এটি হলো "পুরানো দিনের পদ্ধতি"। এখানে দেখা হয় একটি শব্দ কতবার আসছে বা কোন কোন ডকুমেন্টে আছে। এর ভিত্তি হলো পরিসংখ্যান (Statistics)।

- মূল কথা: শব্দ কতবার (Frequency) ব্যবহৃত হয়েছে তার ওপর গুরুত্ব দেওয়া হয়।
- উদাহরণ: ধরো একটি মুভি রিভিউ ডাটাবেজ আছে। "Excellent" শব্দটি অনেকবার এসেছে ভালো মুভিগুলোর ডেসক্রিপশনে। সিস্টেম তখন ধরে নেবে "Excellent" শব্দটি পজিটিভ বা গুরুত্বপূর্ণ।
- সীমাবদ্ধতা: এটি বুঝতে পারে না যে "King" আর "Queen" এর মধ্যে কোনো রাজকীয় সম্পর্ক আছে। এটি কেবল জানে তারা কয়বার এসেছে। (যেমন: TF-IDF, Bag of Words)।

২. Prediction-based Embeddings (প্রেডিকশন-ভিত্তিক)

এটি হলো "আধুনিক ও স্মার্ট পদ্ধতি"। এখানে একটি **Neural Network** মডেলকে ট্রেনিং দেওয়া হয় যাতে সে একটি শব্দের চারপাশের শব্দ দেখে ওই শব্দটি ভবিষ্যদ্বাণী (**Predict**) করতে পারে।

- মূল কথা: একটি শব্দ কোন **Context** বা প্রেক্ষাপটে বসে, তা দেখে এর অর্থ শেখে।
- উদাহরণ (**Word2Vec**): * বাক্য ১: "The **King** is sitting on the throne."
 ○ বাক্য ২: "The **Queen** is sitting on the throne."
 ○ মডেল দেখবে "King" এবং "Queen" দুজনেই "sitting on the throne" এর আগে বসেছে। তাই মডেলের ইনটুইশন তৈরি হবে যে— "King" আর "Queen" এর মধ্যে গভীর কোনো মিল আছে (যেমন দুজনেই রাজকীয় ব্যক্তি)।
- জাদু (**Semantic Relationship**): এই পদ্ধতিতে কম্পিউটার অংক করতে পারে! যেমন: \$King - Man + Woman = Queen\$।

সহজ তুলনা (Table)

বৈশিষ্ট্য	Frequency-based	Prediction-based (Word2Vec/GloVe)

মূল ফোকাস	শব্দ কতবার এসেছে (Statistics)।	শব্দের প্রেক্ষাপট বা মানে (Semantic)।
ভেক্টর সাইজ	সাধারণত বিশাল (Sparse) - অনেক ০ থাকে।	ছোট এবং ঠাসা (Dense) - অনেক তথ্য থাকে।
সম্পর্ক	শব্দের সম্পর্ক বুঝতে পারে না।	রাজার সাথে রানীর কী সম্পর্ক তা বুঝতে পারে।
উদাহরণ	TF-IDF, CountVectorizer	Word2Vec, GloVe

সারকথা:

১. **Frequency-based** হলো "লাইব্রেরিয়ান" যে শুধু গুনে রাখে কোন বই কয়বার ধার নেওয়া হয়েছে।

২. **Prediction-based** হলো "সাহিত্যিক" যে গল্পের প্রেক্ষাপট বুঝে জানে কোন শব্দের সাথে কোন শব্দ মানানসই।

পরীক্ষার জন্য টিপস: যদি প্রশ্ন আসে Word2Vec বা GloVe কোন ক্যাটাগরিতে পড়ে, তবে চোখ বন্ধ করে **Prediction-based** লিখবে।

9. Train/Test Dataset

Definition: Model কে শেখানোর জন্য labeled data ভাগ করা।

মোট Data → 80% Training + 20% Testing

Training Data:

Sentence	Label
I love this movie	Positive
This movie is terrible	Negative
Amazing acting	Positive
Waste of time	Negative

EXAMPLE SENTENCE PROCESSING

6. Train Model: Machine learning models learn patterns.

Common models

- Naive Bayes
- Logistic Regression
- Random Forest
- Deep Learning (LSTM / BERT)

The model learns that: love → positive sentiment

Learning goal

The model learns patterns such as:

- positive words → positive sentiment
- negative words → negative sentiment

This is where the mathematical representation becomes useful for prediction.

10. Model Training (মডেল ট্রেনিং)

Machine learning models যেগুলো NLP এ ব্যবহার হয়:

সহজ → জটিল:

Naive Bayes → সবচেয়ে সহজ, probability based

Logistic Regression → classification এর জন্য ভালো

Random Forest → decision tree এর combination

LSTM → sequence data বোঝে (text এর order)

BERT/GPT → সবচেয়ে শক্তিশালী (Transformer based)

Model কী শেখে?

"love", "amazing", "wonderful" → Positive sentiment

"terrible", "waste", "boring" → Negative sentiment

EXAMPLE SENTENCE PROCESSING

7. Test Model: The trained model predicts sentiment on unseen text.

Input: "I love this movie so much!"

Output: **Sentiment: Positive**

Confidence: **0.94**

Testing checks whether the model generalizes beyond the training examples. In classification tasks, performance depends on the chosen evaluation threshold and the dataset balance.

Test Model

১. মূল উদ্দেশ্য: **Generalization** (সাধারণীকরণ)

মডেলকে আমরা ট্রেনিং ডাটা দিয়ে শিখিয়েছি। এখন টেস্টিং-এর সময় তাকে এমন সব ডাটা দেওয়া হয় যা সে আগে কখনো দেখেনি (**Unseen**)।

- কেন? যাতে আমরা নিশ্চিত হতে পারি যে মডেলটি শুধু উত্তরগুলো "মুখস্থ" (**Overfitting**) করেনি, বরং ভাষার প্যাটার্নটা বুঝতে পেরেছে। একেই বলে **Generalization**।

২. উদাহরণের ব্যবচ্ছেদ

- **Input:** "I love this movie so much!"
- **Model Prediction:** Positive (এটি লেবেল)।
- **Confidence (0.94):** এর মানে হলো মডেল ৯৪% নিশ্চিত যে এটি একটি পজিটিভ কমেন্ট। বাকি ৬% হয়তো সে অন্য কোনো সম্ভাবনা দেখছে। কনফিডেন্স যত বেশি হয়, আমাদের ভরসা তত বাড়ে।

৩. গুরুত্বপূর্ণ দুটি পয়েন্ট (পড়ার জন্য জরুরি)

A. Evaluation Threshold (মূল্যায়ন সীমা)

মডেল যখন কোনো রেজাল্ট দেয়, সে আসলে একটি প্রবাবিলিটি বা সম্ভাবনা দেয় (যেমন ০.৯৪)।

- সাধারণত আমরা একটি সীমা বা **Threshold** ধরি ০.৫। অর্থাৎ ০.৫-এর বেশি হলে 'Positive', কম হলে 'Negative'।
- কিন্তু কিছু ক্ষেত্রে (যেমন ক্যাম্পার ডিটেকশন বা ফ্রড ডিটেকশন) আমরা থ্রেশহোল্ড কমিয়ে ০.৩ বা বাড়িয়ে ০.৮ করতে পারি যাতে ভুল হওয়ার ঝুঁকি কমে। তোমার রেজাল্ট এই থ্রেশহোল্ডের ওপর নির্ভর করে বদলে যেতে পারে।

B. Dataset Balance (ডেটা ভারসাম্য)

মডেল কেমন পারফর্ম করছে তা বোঝার জন্য ডাটা ব্যালেন্স থাকা জরুরি।

- উদাহরণ: ধরো তোমার টেস্ট সেটে ১০০টি মুভি রিভিউ আছে। তার মধ্যে ৯৯টিই পজিটিভ আর ১টি নেগেটিভ। এখন তোমার মডেল যদি চোখ বন্ধ করে সবগুলোকে "Positive" বলে দেয়, তবে সে ৯৯% নির্ভুল (Accuracy) দেখাবে! কিন্তু বাস্তবে মডেলটি আসলে কিছুই শেখেনি, সে শুধু পজিটিভ ডাটায় ডুবে ছিল।
- তাই ভালো টেস্টিং-এর জন্য সব ধরনের ক্যাটাগরির (Spam/Ham বা Positive/Negative) ডাটা সমান বা ভারসাম্যপূর্ণ থাকা উচিত।

TRANSFORMERS AND BERT

Modern NLP is dominated by transformer-based models. The Transformer architecture is based on attention mechanisms and removes recurrence and convolutions. BERT is a bidirectional transformer pretrained with masked language modeling and next sentence prediction, and it can be fine-tuned for downstream NLP tasks.

Why transformers are powerful

- They capture long-range dependencies
- They process sequences more effectively
- They achieve strong results across many NLP tasks

TRANSFORMERS AND BERT

- **Static vs Contextual:** Unlike word2vec, contextual embeddings give each word occurrence a vector depending on its sentence. Allows polysemous words (e.g. "bank" as financial vs river) to have different vectors.
- **ELMo, BERT, GPT, etc.:** Early contextual models used BiLSTMs (ELMo), modern ones use Transformers. BERT (Devlin et al., 2018) is encoder-only, bidirectional Transformer using masked language modeling. GPT is decoder-only (autoregressive).
- **Transformer architecture:** Uses multi-head self-attention to encode all positions together. Key advantages are captures long-range dependencies and parallelizable. (Diagram: simplified transformer block or attention illustration.)
- **BERT specifics:** Pre-trained on huge corpora (English Wikipedia, BooksCorpus). Objective: predict masked tokens and next-sentence. Result: powerful language representations.
- **RoBERTa, DistilBERT, etc.:** Variants with tweaks (no NSP, larger data, distilled smaller models).
- **Use in NLP:** These models provide state-of-art results in many tasks. Often fine-tuned for tasks. Hugging Face Transformers library provides easy access.
- **Trade-offs:** Huge models → high accuracy but heavy compute and memory.

11. Transformers এবং BERT

Transformers এবং **BERT** হলো বর্তমান যুগের AI-এর আসল সুপারস্টার। চ্যাটজিপিটি (ChatGPT) থেকে শুরু করে গুগল ট্রান্সলেট—সবকিছুর পেছনেই এই প্রযুক্তি কাজ করছে। চলো একদম সহজভাবে এর মূল পয়েন্টগুলো বুঝে নেই:

Transformers

- Transformers are modern deep learning models used in NLP
- They are based on the attention mechanism
- They process all words in a sequence together instead of one by one
- They capture long-range relationships between words efficiently

Why Transformers Are Powerful

- Capture long-range dependencies
- Process sequences more effectively
- Achieve strong performance in many NLP tasks

১. Transformers কেন এতো পাওয়ারফুল?

আগেকার মডেলগুলো (RNN/LSTM) একটা বাক্যকে একটার পর একটা শব্দ হিসেবে পড়ত (যেমন আমরা লাইন ধরে পড়ি)। কিন্তু **Transformer** পুরো বাক্য বা প্যারাগ্রাফ একবারে প্রসেস করতে পারে।

- **Long-range dependencies:** একটি বড় বাক্যের শুরুতে থাকা শব্দের সাথে শেষে থাকা শব্দের কী সম্পর্ক, তা সে খুব ভালো বোঝে।
- **Parallelization:** এটি একসাথে অনেক তথ্য প্রসেস করতে পারে, তাই ট্রেনিং অনেক ফাস্ট হয়।
- **Attention Mechanism:** এটি বাক্যের প্রতিটি শব্দের ওপর গুরুত্ব (Attention) দেয় এবং বুঝে নেয় কোন শব্দের সাথে কোনটির সম্পর্ক বেশি।

২. Static vs Contextual Embeddings (খুবই ইম্পোর্ট্যান্ট)

আগের **Word2Vec** ছিল স্ট্যাটিক, আর **BERT** হলো কনটেক্সচুয়াল।

- **Static (Word2Vec):** "Bank" শব্দটির একটিই ভেক্টর বা সংখ্যা ছিল। সে আর্থিক ব্যাংক হোক বা নদীর পাড়—অর্থ এক ই দেখাত।
- **Contextual (BERT):** এটি বাক্যের প্রেক্ষাপট বোঝে।
 - *Sentence A:* "I went to the **bank** to deposit money."
 - *Sentence B:* "I sat on the river **bank**."
 - BERT এই দুটি "bank" শব্দকে আলাদা আলাদা মানে (Vector) দেবে কারণ সে পাশের শব্দগুলো (money vs river) দেখে মানে বোঝে। একেই বলে **Polysemy** বা এক শব্দের বহু অর্থ বোঝা।

BERT

- BERT stands for Bidirectional Encoder Representations from Transformers
- It is a bidirectional transformer model
- It reads text by considering both left and right context
- It is pre-trained and then fine-tuned for different NLP tasks

How BERT is Trained

- Masked Language Modeling: predicts missing words in a sentence
- Next Sentence Prediction: learns relationship between two sentences

৩. BERT (Bidirectional Encoder Representations from Transformers)

গুগলের তৈরি এই মডেলটি পুরো NLP জগত বদলে দিয়েছে।

- **Bidirectional (দ্বিমুখী):** BERT একটি বাক্যের বাম থেকে ডান এবং ডান থেকে বাম—উভয় দিক থেকে শব্দগুলো পড়ে মানে বোঝার চেষ্টা করে।
- **Pre-training (কীভাবে শেখে):** 1. **Masked Language Modeling (MLM):** বাক্যের মাঝখান থেকে একটি শব্দ লুকিয়ে ফেলে (যেমন: "The [MASK] is on the table") এবং মডেলকে সেটি গেস করতে বলে। 2. **Next Sentence Prediction (NSP):** দুটি বাক্য পাশাপাশি দিয়ে জিজ্ঞেস করে—পরের বাক্যটি কি আগেরটির সাথে সামঞ্জস্যপূর্ণ?
- **Fine-tuning:** একবার ট্রেনিং হয়ে গেলে তাকে সামান্য ডাটা দিয়ে স্প্যাম ডিটেকশন বা কোশেন-অ্যানসারিং এর কাজে লাগানো যায়।

৪. BERT বনাম GPT

টেবিলের মতো করে মনে রাখো:

- **BERT:** শুধুমাত্র **Encoder** ব্যবহার করে। এটি টেক্সট বোঝার (Understanding) জন্য সেবা। (যেমন: সার্চ রেজাল্ট, সেন্টিমেন্ট অ্যানালাইসিস)।
- **GPT:** শুধুমাত্র **Decoder** ব্যবহার করে। এটি নতুন টেক্সট বানানোর (Generating) জন্য সেবা। (যেমন: চ্যাটবট, গল্প লেখা)।

৫. Trade-offs (অসুবিধা)

এই মডেলগুলো অনেক বড় (Huge Models) এবং এদের চালাতে অনেক মেমরি ও পাওয়ারফুল গ্রাফিক্স কার্ড (GPU) লাগে। এই সমস্যা সমাধানের জন্য পরে **DistilBERT** (ছোট ও ফাস্ট ভার্সন) বা **RoBERTa**-র মতো মডেল এসেছে।

ATTENTION IN SIMPLE WORDS

Attention helps the model focus on the most relevant words in a sentence.

Example: "The movie was so good that I loved every minute of it."
Attention helps the model understand that good, loved, and every minute carry strong sentiment signals.

Transformers use self-attention to examine the entire sequence at once, which helps them handle long context efficiently.

- **Attention mechanism:** Allows a model to focus on different parts of input sequence when making each prediction. Improves long-range context handling. For example, in translation, attending to relevant source words.
- **Self-Attention:** Special case where input attends to itself. Each token's representation is a weighted sum of all tokens in sequence.
- **Transformers** (Vaswani et al. 2017): Replace recurrence with layers of multi-head self-attention + feed-forward networks. Process all tokens in parallel, capturing global context.
- **Multi-head:** Multiple attention "heads" learn different aspects of word relationships in parallel.
- **Advantages:** Handles long sequences efficiently, allows parallelization (fast training), achieves state-of-art in MT, summarization, etc.
- **Architecture:** Encoder (stack of self-attention + FFN layers) and decoder (with masked attention + encoder-decoder attention). BERT uses only encoder part.
- **Trade-offs:** Very large models requiring GPU/TPU; quadratic complexity in sequence length (attention cost).

12. Attention Mechanism — সহজ ভাষায়

Attention Mechanism হলো আধুনিক AI-এর (যেমন: ChatGPT বা BERT) সেই বিশেষ শক্তি, যা দিয়ে কম্পিউটার মানুষের মতো করে বাক্যের গুরুত্বপূর্ণ অংশগুলো বুঝতে পারে। নিচে সহজ ভাষায় এর প্রতিটি পয়েন্ট বুঝিয়ে দিচ্ছি:

১. সহজ কথায় **Attention** কী?

মানুষ যখন কোনো বাক্য পড়ে, তখন সে সব শব্দের ওপর সমান গুরুত্ব দেয় না। সে শুধু দরকারি শব্দগুলোতে "মনোযোগ" (Attention) দেয়।

- উদাহরণ: "The movie was so **good** that I **loved** every minute of it."
- এখানে মডেল বুঝে নেয় যে, বাক্যটির মূল ভাব বা সেন্টিমেন্ট বোঝার জন্য 'good' এবং 'loved' শব্দ দুটি সবচেয়ে গুরুত্বপূর্ণ। বাকি শব্দগুলো (যেমন: 'The', 'was', 'so') খুব একটা প্রভাব ফেলে না।

২. Self-Attention: নিজের ভেতর সম্পর্ক খোঁজা

এটি ট্রান্সফরমার মডেলের একটি বিশেষ ক্ষমতা। এখানে একটি বাক্যের প্রতিটি শব্দ অন্য সব শব্দের সাথে নিজের সম্পর্ক যাচাই করে দেখে।

- কাজ: প্রতিটি টোকেন (শব্দ) পুরো সিকোয়েন্সের অন্যান্য টোকেনের একটি **Weighted Sum** বা গাণিতিক গড় হিসেবে নিজেকে রিপ্রেজেন্ট করে।
- কেন जरুরি? এটি বাক্যের **Context** বা প্রেক্ষাপট বুঝতে সাহায্য করে। যেমন: "Bank" শব্দটি নদীর পাশে আছে নাকি টাকার সাথে, সেটা সে চারপাশের শব্দের সাথে সম্পর্ক (Self-attention) থেকে বুঝে নেয়।

৩. Multi-head Attention: একসাথে অনেক চোখে দেখা

মডেলটি শুধু একদিকে মনোযোগ দেয় না, বরং অনেকগুলো "Head" বা দৃষ্টি দিয়ে একসাথে বিভিন্ন ধরনের সম্পর্ক খোঁজে।

- উদাহরণ: একটি হেড হয়তো দেখছে ব্যাকরণগত সম্পর্ক (Subject-Verb), আরেকটি হেড হয়তো দেখছে শব্দের অর্থগত মিল। এভাবে সমান্তরালভাবে (Parallel) কাজ করায় মডেল অনেক বেশি শক্তিশালী হয়।

৪. কেন এটি RNN বা LSTM-এর চেয়ে ভালো?

- **Long-range context:** আগেকার মডেলগুলো বাক্য অনেক বড় হলে শুরুর কথা ভুলে যেত। Attention মেকানিজম থাকায় ট্রান্সফরমার বাক্যের শুরু থেকে শেষ পর্যন্ত সবকিছুর সাথে সম্পর্ক বজায় রাখতে পারে।
- **Parallelization:** আগে শব্দগুলো একে একে (Serial) প্রসেস করতে হতো, যা সময়সাপেক্ষ ছিল। ট্রান্সফরমার সব শব্দ একসাথে প্রসেস করতে পারে, তাই ট্রেনিং অনেক দ্রুত হয়।

৫. সীমাবদ্ধতা (Trade-offs)

- **High Complexity:** বাক্যের দৈর্ঘ্য যত বাড়ে, ক্যালকুলেশনের পরিমাণ তার বর্গাকারে (**Quadratic complexity**) বাড়তে থাকে। অর্থাৎ বাক্য ২ গুণ বড় হলে হিসাব হবে ৪ গুণ বেশি।
- **Heavy Resources:** এই মডেলগুলো চালাতে শক্তিশালী **GPU** বা **TPU** লাগে।

সহজ ইনটুইশন:

ধরো তুমি একটি প্যারাগ্রাফ থেকে একটি প্রশ্নের উত্তর খুঁজছো। তুমি পুরো প্যারাগ্রাফের প্রতিটি শব্দ সমানভাবে না পড়ে শুধু সেই লাইনগুলোর ওপর **Attention** দাও যেখানে উত্তরটি থাকার সম্ভাবনা আছে। AI-তে 'Attention' ঠিক এই কাজটিই করে।

TRANSFER LEARNING & FINE-TUNING

- **Transfer Learning in NLP:** Use a model pre-trained on large corpora and adapt it to a specific task. Saves time/data vs training from scratch. Common since BERT (2018) and GPT.
- **Fine-Tuning:** Add task-specific head (e.g. classification layer) and continue training on labeled data. The entire model (or most layers) is updated slightly on the new data. Highly effective for tasks like sentiment or NER.
- **Examples:** BERT fine-tuned on GLUE tasks (e.g. sentiment SST, NLI MNLI). Hugging Face makes this easy. Also feature-based: use embeddings as input to simpler model (less common now).
- **Benefits:** Leverages vast pretraining knowledge (grammar, facts). Often few-shot or one-shot learning possible. E.g., BERT reached 93% accuracy on SST with <10k examples.
- **Pre-trained Libraries:** Hugging Face Transformers, spaCy pretrained pipelines (NER/POS), TensorFlow Hub (ELMo).

13. Transfer Learning এবং Fine-Tuning

Transfer Learning এবং **Fine-tuning** হলো আধুনিক AI-এর সেই জাদু যার কারণে এখন যে কেউ অল্প ডেটা দিয়েও খুব শক্তিশালী মডেল বানিয়ে ফেলতে পারে। সহজ কথায় বললে—এটি হলো অন্য কারো "অভিজ্ঞতা" নিজের কাজে লাগানো।

নিচে পয়েন্টগুলো সহজ করে বুঝিয়ে দিচ্ছি:

১. Transfer Learning (একজনের জ্ঞান অন্যজনে স্থানান্তর)

আগে যখন আমরা কোনো মডেল বানাতাম, তাকে একদম শুরু (Scratch) থেকে হাতে-খড়ি দিতে হতো। কিন্তু এখন আমরা এমন মডেল ব্যবহার করি যা আগে থেকেই অনেক বড় ডাটা (যেমন: উইকিপিডিয়া বা কোটি কোটি বই) পড়ে ভাষা শিখে নিয়েছে।

- সহজ উদাহরণ: একজন মানুষ যে সাইকেল চালাতে জানে, তার জন্য মোটরসাইকেল চালানো শেখা অনেক সহজ। এখানে "ব্যালেন্স করার জ্ঞান" হলো Transfer Learning।
- কেন দরকার? এতে সময় বাঁচে এবং অনেক কম ডাটা দিয়েও দারুণ কাজ করা যায়।

২. Fine-Tuning (বিশেষ কাজের জন্য ঝালাই করা)

BERT বা GPT হলো "জ্যাক অফ অল ট্রেডস" (সব কাজের কাজী)। তারা ভাষা চেনে, কিন্তু তোমার নির্দিষ্ট কাজ (যেমন: স্প্যাম ডিটেকশন) হয়তো তারা স্পেশালি জানে না।

- কীভাবে করে: আমরা প্রি-ট্রেন্ড মডেলটির মাথার ওপর একটি ছোট নতুন লেয়ার (Classification Layer) যোগ করি। এরপর তোমার কাছে থাকা স্পেসিফিক লেবেলড ডাটা দিয়ে মডেলটিকে হালকা করে আবার ট্রেনিং দেওয়া হয়।
- ফলাফল: পুরো মডেলটির ভেতরের জ্ঞান সামান্য আপডেট হয় তোমার নির্দিষ্ট টাস্কটি আরও নির্ভুলভাবে করার জন্য।

৩. কেন এটি এত শক্তিশালী? (Benefits)

- **Vast Knowledge:** মডেলটি আগে থেকেই গ্রামার, শব্দের মানে এবং সাধারণ জ্ঞান (Facts) জানে।
- **Few-shot Learning:** খুব সামান্য ডাটা (যেমন ১০ হাজারেরও কম উদাহরণ) দিয়েও এটি ৯৩-৯৫% নির্ভুলতা দিতে পারে।
- **উদাহরণ:** BERT মডেলকে GLUE (এক সেট NLP চ্যালেঞ্জ) দিয়ে ফাইন-টিউন করে অনেক কঠিন লজিক্যাল সমস্যার সমাধান করা যায়।

৪. টুলস এবং লাইব্রেরি

এখন আর এই বড় বড় মডেল কেউ নিজের কম্পিউটারে স্ক্র্যাচ থেকে ট্রেনিং করায় না। সবাই এই লাইব্রেরিগুলো ব্যবহার করে:

- **Hugging Face Transformers:** এটি বর্তমানে সবচেয়ে জনপ্রিয় প্ল্যাটফর্ম যেখান থেকে তুমি BERT, GPT, বা RoBERTa-র মতো হাজার হাজার মডেল ডাউনলোড করে কয়েক লাইনেই ফাইন-টিউন করতে পারো।
- **spaCy:** এদের আগে থেকেই তৈরি করা পাইপলাইন (NER, POS tagging) আছে যা সরাসরি ব্যবহার করা যায়।

৫. একটি বাস্তব উদাহরণ

ধরা যাক, তুমি একটি মডেল বানাতে যা ব্যাংকের ইমেইল দেখে বলবে এটি "অভিযোগ" নাকি "ধন্যবাদ"।

1. **Transfer Learning:** তুমি গুগলের প্রি-ট্রেন্ড BERT মডেলটি নিলে (সে ইংরেজি ভাষা চেনে)।
2. **Fine-Tuning:** তুমি তোমার ব্যাংকের ৫০০টি ইমেইল দিয়ে তাকে একটু ট্রেন করালে।
3. **Result:** এখন তোমার মডেলটি খুব সহজেই "ব্যাংক" কনটেক্সটে ইমেইলগুলো আলাদা করতে পারবে, কারণ সে ভাষা আগে থেকেই জানত, তুমি শুধু তাকে "ব্যাংকের ভাষা" চিনিয়ে দিলে।

সারকথা: **Transfer Learning** হলো একটা শিক্ষিত মানুষকে হায়ার করা, আর **Fine-Tuning** হলো তাকে তোমার অফিসের নিয়মকানুন বুঝিয়ে দেওয়া।

সুবিধা: ✓ কম data দরকার হয় ✓ কম সময় লাগে

✓ Better accuracy পাওয়া যায় (BERT, মাত্র ১০,০০০ example দিয়ে ৯৩% accuracy!)

EVALUATION METRICS

To judge NLP models, we use different metrics depending on the task.

For classification

- Accuracy
- Precision
- Recall
- F1 score

For generation tasks

- BLEU for translation
- ROUGE for summarization

14. Evaluation Metrics (কীভাবে model measure করবো?)

Classification এর জন্য:

Confusion Matrix (উদাহরণ):

	Predicted	
	Positive	Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

TP = True Positive (Positive বলেছে, আসলেও Positive)

TN = True Negative (Negative বলেছে, আসলেও Negative)

FP = False Positive (Positive বলেছে, আসলে Negative)

FN = False Negative (Negative বলেছে, আসলে Positive)

১. Accuracy (নির্ভুলতা)

মোট কতগুলো প্রেডিকশনের মধ্যে কতগুলো সঠিক হয়েছে।

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

২. Precision (সতর্কতা)

মডেল যেগুলোকে "Positive" বলেছে, সেগুলোর মধ্যে আসলে কয়টি সত্যি পজিটিভ। (এটি ভুল পজিটিভ বা FP কমাতে সাহায্য করে)।

$$\text{Precision} = \frac{TP}{TP + FP}$$

৩. Recall (খুঁজে বের করার ক্ষমতা)

বাস্তবে যতগুলো "Positive" ছিল, তার মধ্যে মডেল কয়টি সঠিকভাবে খুঁজে বের করতে পেরেছে। (এটি FN কমাতে সাহায্য করে)।

$$\text{Recall} = \frac{TP}{TP + FN}$$

৪. F1 Score (ভারসাম্য)

যখন Precision এবং Recall দুটোর মধ্যেই ব্যালেন্স দরকার হয়, তখন F1 Score ব্যবহার করা হয়। এটি এই দুটির **Harmonic Mean**।

$$F1 \text{ Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

সহজ উদাহরণ:

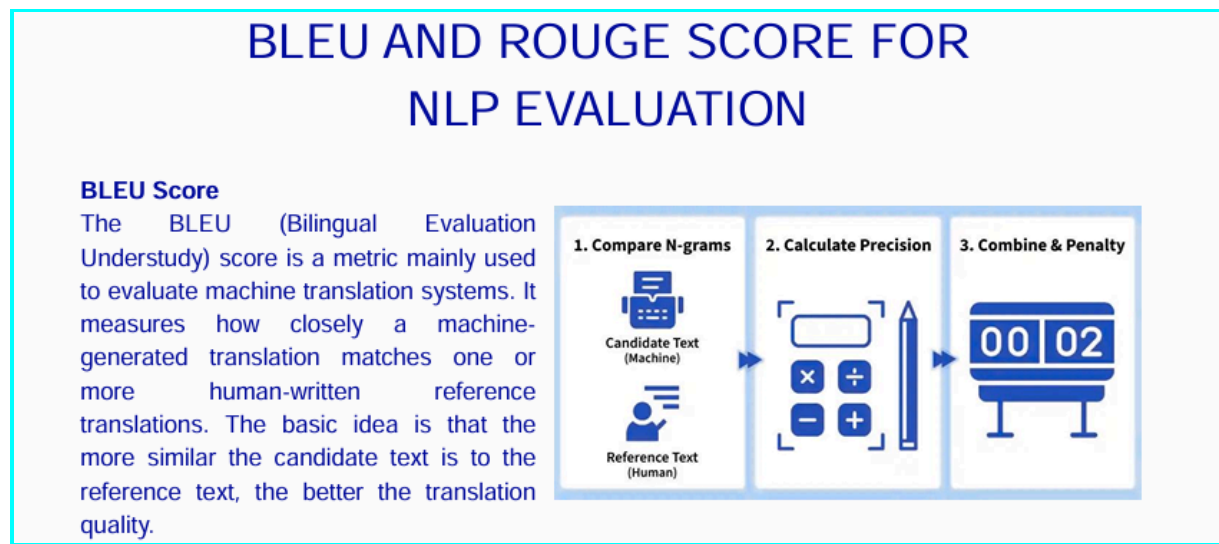
ধরো ১০০ জন মানুষের মধ্যে ১০ জনের করোনা আছে।

- তোমার মডেল ১০ জনকেই "করোনা আছে" বলল, কিন্তু তাদের মধ্যে আসলে ৫ জনের করোনা ছিল।
- এখানে **Precision** কম হবে কারণ মডেল অনেক সুস্থ মানুষকে ভুল করে রোগী বলেছে।
- কিন্তু **Recall** বেশি হবে কারণ সে আসল রোগীদের কাউকেই মিস করেনি।

একটি ছোট টিপস: যদি তোমার ডেটাসেটে পজিটিভ আর নেগেটিভ ডাটার সংখ্যায় অনেক পার্থক্য থাকে (যেমন ৯৯টি সুস্থ, ১টি রোগী), তবে শুধু **Accuracy** দেখে মডেল বিচার করা ঠিক নয়, তখন **F1 Score** দেখা সবচেয়ে বুদ্ধিমানের কাজ।

Generation Tasks এর জন্য:

মেশিন লার্নিং বা AI যখন কোনো টেক্সট জেনারেট করে (যেমন: অনুবাদ করা বা সামারি করা), তখন সেই আউটপুটটি কতটা ভালো হয়েছে তা মাপার জন্য **BLEU** এবং **ROUGE** স্কোর ব্যবহার করা হয়। সহজ ভাষায় এদের কাজ হলো মেশিনের তৈরি টেক্সটকে মানুষের লেখা টেক্সটের সাথে তুলনা করা।



BLEU Score:

Full form: Bilingual Evaluation Understudy

Use: Machine Translation evaluate করতে।

BLEU Score (Bilingual Evaluation Understudy)

এটি মূলত **Machine Translation** (অনুবাদ) মূল্যায়নের জন্য ব্যবহৃত হয়।

- **মূল মন্ত্র (Precision):** মেশিন যে অনুবাদটি করল, তার শব্দগুলো মানুষের দেওয়া রেফারেন্স অনুবাদের সাথে কতটা মিলছে?
- কীভাবে কাজ করে: এটি **n-gram overlap** দেখে। অর্থাৎ ১টি শব্দ (unigram), ২টি শব্দ (bigram) বা ৩টি শব্দের জোড়া মানুষের অনুবাদের সাথে মিলছে কি না।
- **Brevity Penalty:** মেশিন যদি খুব ছোট অনুবাদ করে পার পাওয়ার চেষ্টা করে, তবে BLEU স্কোর তাকে পেনাল্টি বা জরিমানা দিয়ে স্কোর কমিয়ে দেয়।

কীভাবে কাজ করে:

Reference (Human): "The cat sat on the mat"

Candidate (Machine): "The cat is on the mat"

↓ n-gram overlap দেখো ↓

1-gram match: The ✓ cat ✓ is ✗ on ✓ the ✓ mat ✓ → 5/6

2-gram: "The cat" ✓ "cat is" ✗ "is on" ✗ "on the" ✓ "the mat" ✓ → 3/5

BLEU = এই সব combine করে একটা score → 0 থেকে 1
(বেশি হলে ভালো)

Short output penalty (Brevity Penalty):

"cat mat" → সব word match করলেও খুব ছোট

→ BLEU penalty দেয় কারণ reference এর চেয়ে অনেক ছোট

BLEU AND ROUGE SCORE FOR NLP EVALUATION

ROUGE Score

The ROUGE (Recall Oriented Understudy for Gisting Evaluation) score is mainly used to evaluate text summarization and other text generation tasks. It measures how much of the important information from the reference text is captured in the generated output. The final ROUGE score ranges from 0 to 1, where higher values indicate better content coverage and similarity to the reference text.

Unlike BLEU, which focuses more on precision, ROUGE emphasizes recall, meaning it checks how much relevant content is covered.

Machine Summary (Candidate)

Human Reference

ROUGE: Emphasizes Recall (What was remembered?)

Measures: How much of the original meaning is in the generated summary? (Higher is better coverage)

ROUGE Score:

Full form: Recall Oriented Understudy for Gisting Evaluation

Use: Text Summarization evaluate করতে।

ROUGE Score (Recall Oriented Understudy for Gisting Evaluation)

এটি মূলত **Text Summarization** (সারসংক্ষেপ) মূল্যায়নের জন্য ব্যবহৃত হয়।

- মূল মন্ত্র (**Recall**): মানুষের লেখা সামারিতে যে গুরুত্বপূর্ণ তথ্যগুলো ছিল, তার কতটুকু মেশিনের সামারিতে উঠে এসেছে?
- কেন **Recall**? সামারির ক্ষেত্রে জরুরি তথ্য বাদ পড়া মানেই বড় ভুল, তাই এখানে Recall বা কতটুকু কভার করল সেটা বেশি গুরুত্বপূর্ণ।

Key difference থেকে BLEU:

BLEU → Precision focus (তুমি যা বললে তার কতটা সঠিক?)

ROUGE → Recall focus (reference এর কতটুকু তুমি cover করলে?)

Example:

Reference: "The cat sat on the mat near the door"

Summary: "The cat sat on the mat"

ROUGE-1 Recall = matched words / reference words
= 6/9 = 0.67

BLEU vs ROUGE একসাথে দেখো:

Aspect	BLEU	ROUGE
Focus	Precision	Recall
Main Use	Translation	Summarization
Method	n-gram precision	n-gram recall
Short text	Brevity penalty	No strict penalty
Score Range	0 to 1	0 to 1

CHALLENGES IN NLP

NLP is powerful, but language is complex.

Challenges

- Ambiguity
- Sarcasm
- Misspellings
- Multilingual text
- Bias in data
- High computation cost for large models

Research has shown that language models and embeddings can carry and amplify bias, and privacy-sensitive text often requires anonymization or pseudonymization before use. Quantization is one way to reduce inference cost for transformer models.

15. Challenges in NLP (সমস্যাগুলো)

NLP শক্তিশালী হলেও মানুষের ভাষার জটিলতার কারণে কিছু বড় চ্যালেঞ্জের সম্মুখীন হয়। স্লাইডে দেওয়া পয়েন্টগুলো নিচে সহজভাবে বুঝিয়ে দিচ্ছি:

১. Ambiguity (অস্পষ্টতা)

একটি শব্দের বা বাক্যের একাধিক অর্থ হতে পারে। কম্পিউটার বুঝতে পারে না কোন প্রেক্ষাপটে কোনটি সঠিক।

- উদাহরণ: "I saw a bat." — এখানে 'bat' কি খেলার ব্যাট নাকি বাদুড়? এটি বোঝা কম্পিউটারের জন্য কঠিন।

২. Sarcasm (ব্যঙ্গ বা শ্লেষ)

মানুষ যখন মনে যা আছে তার উল্টোটা বলে মজা করে, তখন AI সেটা ধরতে পারে না।

- উদাহরণ: খুব বাজে সার্ভিস পাওয়ার পর কেউ যদি বলে— "Wow, what a great service!" —মডেল হয়তো 'great' দেখে ভাববে এটি পজিটিভ, কিন্তু বাস্তবে এটি নেগেটিভ।

৩. Misspellings and Slang (বানান ভুল ও আঞ্চলিক ভাষা)

সোশ্যাল মিডিয়ায় আমরা অনেক ভুল বানান বা সংক্ষিপ্ত রূপ (যেমন: 'u' এর বদলে 'you', 'lol') ব্যবহার করি। এগুলো স্ট্যান্ডার্ড ডিকশনারিতে না থাকায় মডেল বিভ্রান্ত হয়।

৪. Multilingual and Bias (বহুভাষা ও পক্ষপাত)

- **Multilingual:** সব ভাষার জন্য সমান পরিমাণ ডাটা ইন্টারনেটে নেই (যেমন ইংরেজির তুলনায় বাংলা ডাটা কম), তাই মডেল সব ভাষায় সমান দক্ষ হয় না।
- **Bias:** যদি ট্রেনিং ডাটায় কোনো বিশেষ ধর্ম বা লিঙ্গের প্রতি ঘৃণা থাকে, তবে মডেলও সেই পক্ষপাতিত্ব শিখে ফেলে।

৫. টেকনিক্যাল চ্যালেঞ্জ ও সমাধান (Advanced)

- **High Computation Cost:** BERT বা GPT-এর মতো বড় মডেল চালাতে প্রচুর বিদ্যুৎ ও মেমরি লাগে।
 - সমাধান (**Quantization**): এটি একটি টেকনিক যেখানে মডেলের বড় সংখ্যাগুলোকে (Weights) ছোট করা হয় যাতে সে কম মেমরিতেও দ্রুত কাজ করতে পারে।
- **Privacy and Anonymization:** মানুষের ব্যক্তিগত তথ্য (যেমন ফোন নম্বর বা ইমেইল) নিরাপদ রাখার জন্য **Anonymization** (নাম-ঠিকানা মুছে ফেলা) বা **Pseudonymization** (ছদ্মনাম ব্যবহার) করা হয়।

সারকথা:

NLP-এর মূল লড়াই হলো ভাষার সূক্ষ্ম অনুভূতি (যেমন ব্যঙ্গ) এবং মানুষের ভুলগুলো বুঝতে পারা, আর বিশাল মডেলগুলোকে সাশ্রয়ী (Cost-effective) করে তোলা।

Summary — সব একসাথে মনে রাখো

NLP Pipeline মনে রাখার shortcut:

Raw Text

- ↓ Preprocessing (clean করো)
- ↓ Tokenization (ভাগ করো)
- ↓ Representation (BoW / TF-IDF / Embedding)
- ↓ Train/Test Split (80/20)

- ↓ Model Train (Naive Bayes থেকে BERT)
- ↓ Evaluate (Accuracy, F1, BLEU, ROUGE)

Key takeaway:

NLP = Linguistic knowledge + Math/Statistics + Machine Learning + Deep Learning (Transformers)

=